

Documentazione PagamentoService

Scopo della classe

PagamentoService gestisce la logica di business relativa ai transazioni fiscali e ai pagamenti degli utenti (entità **Pagamento**). Estende **AbstractService<Pagamento, PagamentoDto>**, ereditando le operazioni CRUD trasversali e mettendo a disposizione metodi di ricerca e filtraggio avanzati basati su utente, stato della transazione e intervalli temporali.

Dipendenze

- **PagamentoMapper**: gestisce la conversione tra l'entità **Pagamento** e la sua rappresentazione **PagamentoDto** (compresi i metodi di conversione di liste).
- **PagamentoRepository**: interroga la base dati fornendo metodi di ricerca personalizzati per utente, stato e intervalli di date.

Costruttore

Riceve tramite Dependency Injection il repository generico, il converter, il mapper specifico (**PagamentoMapper**) e il repository specifico (**PagamentoRepository**). Richiama **super(repository, converter)** per inizializzare la superclasse e salva le dipendenze nei campi locali della classe.

Metodi

getPagamentoPerUtentId(Integer utentId)

Ricerca lo storico di tutti i pagamenti effettuati da un singolo utente specificato dal suo identificativo univoco (**utenteId**). Il repository esegue la query **findByUtenteId(utenteId)** e la lista di entità ottenuta viene convertita in DTO tramite **pagamentoMapper.toDTOList()**.

getPagamentiPerStato(PagamentoStatoEnum stato)

Filtra i pagamenti registrati a sistema in base al loro stato (es. **COMPLETATO**, **IN_ATTESA**, **FALLITO**) definito nell'enumerazione **PagamentoStatoEnum**. Il repository interroga la base dati con **findByStato(stato)** e restituisce la lista trasformata in **PagamentoDto**.

getPagamentiPerGiorno(LocalDate giorno)

Recupera tutti i pagamenti effettuati in un determinato giorno solare. Il metodo calcola l'intervallo temporale completo convertendo la data in un range **LocalDateTime** compreso tra l'inizio del giorno (**00:00:00** via **giorno.atStartOfDay()**) e la fine del giorno (**23:59:59.999999999** tramite **giorno.atTime(LocalTime.MAX)**). Interroga quindi il repository con **findByDataPagamentoBetween(inizio, fine)** e converte i risultati in DTO.

getPagamenti(Integer userId, PagamentoStatoEnum stato, LocalDate giorno)

Fornisce un metodo di ricerca flessibile a cascata per filtrare i pagamenti in base ai parametri forniti in input:

- Se viene fornito **userId**, recupera i pagamenti per quell'utente.
- Altrimenti, se è specificato lo **stato**, filtra i pagamenti in base allo stato indicato.

- Altrimenti, se è indicata una data (**giorno**), calcola il range di inizio e fine giornata ed esegue la ricerca per intervallo temporale.
- Se nessun parametro viene passato (tutti **null**), recupera l'intero elenco di tutti i pagamenti presenti nel database tramite **findAll()**.

Infine, la lista di entità risultante viene convertita in una lista di **PagamentoDto** tramite il mapper.

Flusso di esecuzione

Controller → PagamentoService → PagamentoRepository → Database → List<Pagamento> → PagamentoMapper → List<PagamentoDto> → Controller → JSON.

Vantaggi

Separazione delle responsabilità, supporto a ricerche multicriterio flessibili, gestione rigorosa delle date con range orari per evitare perdite di transazioni, conversione Entity/DTO centralizzata e codice facilmente manutenibile.